

# LLVM Function Inlining with Dead Code Elimination

ML Varshitha - 1RV23CS121

Nayana Jagadish Raikar - 1RV23CS149

Nidhi Vinayak Kulkarni - 1RV23CS151

Faculty In-charge: Dr. Deepika Dash

2026

# Project Overview

## Goal:

Build an LLVM `ModulePass` that:

- Selectively inlines small functions
- Uses a cost heuristic for optimization
- Removes dead functions after inlining
- Eliminates unreachable basic blocks

## Main Idea:

Improve runtime efficiency while reducing unnecessary function call overhead.

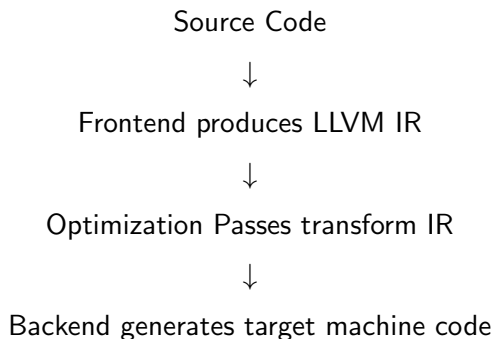
# What is LLVM?

**LLVM** is a compiler infrastructure used to build optimizers, code generators, and analysis tools.

- It represents programs using **LLVM Intermediate Representation (IR)**.
- IR is platform-independent, strongly typed, and easier to analyze than machine code.
- Optimization passes transform IR before final machine code generation.
- LLVM allows custom passes to be added into the compiler pipeline.

**Why it matters:** Optimizations can be implemented once on IR and reused for many target architectures.

# How LLVM Compilation Works



Our project works in the **optimization pass stage**, where the IR is improved before code generation.

# LLVM IR and Optimization Passes

- LLVM IR stores program logic as functions, basic blocks, and instructions.
- A pass scans this IR and applies a specific transformation or analysis.
- Function inlining replaces selected function calls with the callee body.
- Dead code elimination removes code that is no longer useful after transformation.

## Project Connection

Our pass reads LLVM IR, identifies safe and beneficial inlining opportunities, performs the transformation, and then cleans up unused functions and unreachable blocks.

# Why Function Inlining?

## Without Inlining

- Extra function call overhead
- Stack push/pop operations
- Frequent branching
- Lower optimization opportunities

## With Inlining

- Direct code substitution
- Faster execution
- Better compiler optimization
- Reduced call overhead

Caller + Callee  $\rightarrow$  Single Optimized Block

# System Workflow

- ➊ **Input:** Takes LLVM IR from compiled C/C++ code.
- ➋ **Traverse:** Iterates each function and its call instructions.
- ➌ **Analyze:** Counts instructions, finds call sites, detects recursion via CallGraph BFS.
- ➍ **Compute Cost:** Applies heuristic — `cost = instructions × call_sites`.
- ➎ **Inline:** Uses `InlineFunction()` at every approved call site (`cost < 45`).
- ➏ **Clean Up:** Removes dead functions and unreachable blocks in a loop until stable.
- ➐ **Output:** Final IR is smaller, cleaner, and easier for later compiler stages.

The pass computes a lightweight estimate:

$$\text{cost} = \text{instructions} \times \text{call\_sites}$$

- **Instruction count** estimates how much code will be copied.
- **Call-site count** estimates how many times that copying may happen.
- Functions are inlined only if  $\text{cost} < 45$ .
- This avoids aggressive inlining that may increase binary size too much.



# Pass Phases

- ➊ **Analysis Phase:** Detect recursion via CallGraph BFS, count instructions, and record call sites.
- ➋ **Decision Phase:** Compare heuristic cost with the threshold (45) and reject risky functions.
- ➌ **Inlining Phase:** Use LLVM `InlineFunction()` to replace approved calls.
- ➍ **Dead Function Removal:** Delete functions that have no remaining callers.
- ➎ **Cleanup Phase:** Remove unreachable basic blocks created by transformation.

Each phase is ordered carefully because inlining can create new dead code that should be removed only after the transformation is complete.

# Iterative Dead Function Removal

## Why One Pass Is Not Enough

After inlining, a function may still appear to have callers — because those callers are themselves dead functions not yet deleted.

**Example:** `chain_inline`

- ➊ After inlining, `@funcC` has no callers → deleted.
- ➋ Now `@funcB` has no callers → deleted.
- ➌ Now `@funcA` has no callers → deleted.
- ➍ Loop exits: no more dead functions found.

## Implementation

The deletion phase runs in a `do { ... } while (changed)` loop, repeating until a full scan finds zero dead functions.

# Core LLVM APIs Used

- PassInfoMixin<T>
- InlineFunction()
- InlineFunctionInfo
- CallGraph
- CallGraphNode
- CallBase
- Function
- BasicBlock
- SmallVector
- SmallPtrSet

## Important Note

The implementation uses the **New Pass Manager** (PassInfoMixin), registered via `llvmGetPassPluginInfo()` and invoked with `-passes="inline-dce"`.

# Handling Special Cases

## Recursive Functions

Recursive functions — including mutually recursive ones — are detected using a CallGraph worklist BFS and blocked before cost analysis to avoid infinite inlining.

## Large Functions

Functions whose cost (`instructions`  $\times$  `call_sites`) meets or exceeds the threshold of 45 are not inlined.

## Dead Code Cleanup

Unused functions and unreachable blocks are safely removed after all inlining is complete. The deletion loop repeats until no more dead functions remain.

# Example Transformation

## Before Optimization

```
define i32 @add(i32 %a, i32 %b) {  
    %1 = add i32 %a, %b  
    ret i32 %1  
}  
call i32 @add(i32 2, i32 3)
```

## After Optimization

```
%1 = add i32 2, 3
```

Function call overhead is removed completely.

# Test Cases

| Test             | Before  | After   | Result                  |
|------------------|---------|---------|-------------------------|
| small_func       | 2 funcs | 1 func  | Inlined                 |
| large_func       | 2 funcs | 2 funcs | Skipped                 |
| recursive_func   | 2 funcs | 2 funcs | Blocked                 |
| multi_call       | 2 funcs | 1 func  | Fully inlined (5 sites) |
| mixed            | 4 funcs | 3 funcs | Partial inline          |
| single_use       | 2 funcs | 1 func  | Inlined                 |
| multi_func       | 3 funcs | 1 func  | Both inlined            |
| mutual_recursive | 3 funcs | 3 funcs | Blocked (cycle)         |
| mixed_recursive  | 3 funcs | 2 funcs | Partial inline          |
| chain_inline     | 4 funcs | 1 func  | Chain collapsed         |

All **10/10** tests passed successfully.

# Quantified Space Savings

## IR Line Count Reduction

| Test           | Before | After | Saved |
|----------------|--------|-------|-------|
| small_func     | 12     | 7     | 42%   |
| large_func     | 70     | 70    | 0%    |
| recursive_func | 20     | 20    | 0%    |
| multi_call     | 18     | 11    | 39%   |
| mixed          | 85     | 77    | 9%    |

**Functions Deleted:** 3 total  
(@add, @square, @tiny)

## Call Instructions Eliminated

| Test         | Calls Removed |
|--------------|---------------|
| small_func   | 1             |
| multi_call   | 5             |
| mixed        | 1             |
| <b>Total</b> | <b>7</b>      |

## CPU cycles saved per run:

Each eliminated call saves  $\approx 8$ –20 cycles

(prologue + epilogue + jump)

**Total:**  $\approx 56$ –140 cycles saved

# Threshold Sensitivity Analysis

Effect of changing `INLINE_THRESHOLD` on inlining decisions:

| Threshold | @add<br>cost 2 | @square<br>cost 10 | @big<br>cost 48 | @heavy<br>cost 57 |
|-----------|----------------|--------------------|-----------------|-------------------|
| 1         | Skip           | Skip               | Skip            | Skip              |
| 5         | Inline         | Skip               | Skip            | Skip              |
| 15        | Inline         | Inline             | Skip            | Skip              |
| <b>45</b> | <b>Inline</b>  | <b>Inline</b>      | <b>Skip</b>     | <b>Skip</b>       |
| 55        | Inline         | Inline             | Inline!         | Skip              |
| 100       | Inline         | Inline             | Inline!         | Inline!           |

**Why 45?** @big (cost 48) is skipped; at 55 it would be inlined.



# Our Pass vs LLVM Built-in always-inline

| Test           | Our pass<br>funcs | Built-in<br>funcs | Our advantage                          |
|----------------|-------------------|-------------------|----------------------------------------|
| small_func     | 1                 | 2                 | Inlines @add; built-in does not        |
| large_func     | 2                 | 2                 | Both skip correctly                    |
| recursive_func | 2                 | 2                 | Both block correctly                   |
| multi_call     | 1                 | 2                 | Inlines all 5 sites; built-in does not |
| mixed          | 3                 | 4                 | Inlines @tiny; built-in does not       |

## Why the built-in does nothing

LLVM's `-always-inline` only inlines functions marked with `alwaysinline`. Our pass applies cost-based inlining to eligible functions without annotations.

# Project Structure

- `src/InlinePass.cpp`
  - Main pass implementation
- `tests/`
  - 10 LLVM IR test cases
- `docs/`
  - Design, Implementation, Evaluation, Screenshots
- `static/index.html + app.py`
  - Flask web UI for IR visualisation
- Build scripts: `build.sh`, `run.sh`

# Challenges Faced

- LLVM New Pass Manager API compatibility across versions
- Iterator safety during IR modification (collect-then-modify pattern)
- Detecting mutual recursion via CallGraph BFS (not just direct self-calls)
- Preventing excessive code expansion with a well-tuned threshold
- Maintaining SSA correctness and CFG integrity after transformations

## Performance Improvements

- Reduced function call overhead
- Cleaner LLVM IR
- Improved optimization opportunities

## Compiler Benefits

- Modular optimization pipeline
- Efficient dead code cleanup
- Better intermediate representation

IR line count reduced by up to **73%** (`chain_inline`: 26  $\rightarrow$  7 lines) compared to **0%** reduction with LLVM's `-always-inline` on the same files.

# Future Enhancements

- Advanced cost modeling (loop presence, call depth heuristic)
- Profile-guided inlining using runtime profiling data
- Machine learning based heuristics for threshold selection
- Support for indirect calls and virtual dispatch
- Integration with LLVM's full optimization pipeline (-O2/-O3)

# Conclusion

- Implemented selective LLVM function inlining using the New Pass Manager
- Added dead function elimination via use-def chain analysis
- Removed unreachable basic blocks after transformation
- Used heuristic-driven optimization ( $\text{cost} = \text{instrs} \times \text{calls}$ , threshold 45)
- Detected mutual and direct recursion via CallGraph BFS
- Successfully validated across all 10 test cases with 10/10 PASS

Compiler Optimizations Improve Both  
Performance and Code Quality

# Thank You

Questions?